

Instrucciones

- Lea con detenimiento y desarrolle **individualmente** cada una de las actividades a realizar durante la experiencia.
- Cree un archivo comprimido con extensión **.zip**, **.rar** o **.tar.gz** que incluya todos los archivos con extensión **.cpp** y **.h** utilizados para el desarrollo. El nombre del archivo debe tener el siguiente formato: **TEL102_P3_Nombre_Apellido.zip** (Ej. TEL102_P3_Patricio_Olivares.zip), sin incluir tildes.
- Enviar el archivo a través de la página de aula del ramo, sección “Práctico 3” hasta las 23:59:59 del día Viernes 25/10/2024 hora local continental de Chile.
- Trate de utilizar herramientas conocidas o aprendidas en clases. **No copie literalmente de recursos online.**
- Sea riguroso con las instrucciones de desarrollo. El no cumplimiento de las instrucciones tanto de entrega como de desarrollo se traducirá en descuentos en el puntaje obtenido.
- ¡Éxito!

1. Hospital TeleVital

El hospital **TeleVital** está desarrollando un sistema de gestión para optimizar la reserva de consultas médicas. Para ello, han solicitado sus servicios como programador.

Su tarea es crear un programa utilizando el paradigma de **Programación Orientada a Objetos (POO)** en C++. El programa debe permitir tanto el registro como la gestión eficiente de las consultas médicas. Recuerde que, al aplicar POO, la definición de la clase debe ir en un archivo **.h**, mientras que la implementación de sus métodos debe realizarse en un archivo **.cpp**.

Para el desarrollo, debe utilizar como base los siguientes archivos disponibles en la plataforma Aula de la asignatura:

- `main.cpp`
- `funciones.h`
- `funciones.cpp`

Estos archivos contienen la lógica principal del programa y algunas funciones ya implementadas. Su código **debe ser compatible** y funcionar correctamente en conjunto con el código entregado.

1. (3pts) Defina e implemente la clase abstracta **Consulta**, la cual debe contener los siguientes **atributos privados**:

- **Nombre del paciente:** Almacena el nombre del paciente que solicita la consulta.
- **Especialidad médica:** Indica la especialidad del médico que atenderá la consulta.
- **Fecha de la consulta:** Almacena la fecha en la que está programada la consulta médica.
- **Estado de la reserva:** Un valor booleano que indica si la reserva ha sido confirmada.

Además, la clase debe incluir:

- Un **constructor** que inicialice todos los atributos mencionados.
- Un método `void confirmarReserva()` que cambie el estado de la reserva a confirmada y muestre un mensaje de confirmación.
- Un método virtual puro `void mostrarDetalles() = 0;` que garantice que la clase sea abstracta. Recuerde que los métodos virtuales puros **no deben implementarse en la clase base, solo definirse**.

Puede agregar otros métodos y funcionalidades que considere útiles para el correcto funcionamiento de la clase.

2. (2pts) Defina e implemente la clase **ConsultaGeneral**, la cual hereda de la clase **Consulta** y representa consultas médicas generales. La clase debe cumplir con los siguientes requisitos:

- Agregar un atributo privado adicional para almacenar el nombre del médico que atenderá la consulta.
- Incluir un **constructor** que inicialice tanto los atributos heredados como el nuevo atributo específico.
- Sobrescribir e implementar el método virtual puro `void mostrarDetalles()`, previamente declarado en la clase base **Consulta**. Este método debe mostrar por consola toda la información relevante de la consulta, incluyendo los atributos heredados de **Consulta** y el nuevo atributo de **ConsultaGeneral**.

Puede agregar otros métodos y funcionalidades que considere útiles para el correcto funcionamiento de la clase.

3. (2pts) Defina e implemente la clase **ConsultaEspecializada**, que hereda de la clase **Consulta** y representa consultas médicas de especialidad, las cuales incluyen un costo adicional. La clase debe cumplir con los siguientes requisitos:

- Agregar un atributo privado para almacenar el costo adicional de la consulta.
- Incluir un **constructor** que inicialice tanto los atributos heredados como el nuevo atributo específico.
- Sobrescribir e implementar el método virtual puro `void mostrarDetalles()`, definido en la clase base **Consulta**. Este método debe mostrar en consola toda la información relevante de la consulta, incluyendo los atributos heredados de **Consulta** y el nuevo atributo de **ConsultaEspecializada**.

Puede agregar otros métodos y funcionalidades que considere útiles para el correcto funcionamiento de la clase.

4. (3pts) Integre su código con el código de los archivos entregados y verifique el correcto funcionamiento del programa completo.